Informe de Trabajo — Examen Parcial de Estructuras de Datos

Equipo 1: Farmacia 'FarmaVida'

Integrante	Método asignado
Jorge Luis Reyes Pérez	`MedicamentosAgotados()`
Geonel Sánchez	`PromedioPrecioPorCategoria(String)`
Merlyn Sosa Borges	`ActualizarStock(String, int)`
Yaisel Pinos Durán	`ListarPreciosInversos()`
Ray Sanjudo Mesa	`AplicarAumentoGeneral(double)`

1. Introducción

El presente informe documenta el proceso de análisis, diseño, razonamiento e implementación del Ejercicio 1 del Examen Parcial de Estructuras de Datos (Curso 2026-2027). El equipo 1 tiene asignada la problemática de la farmacia **'FarmaVida'**, y cada integrante es responsable de un método específico dentro de la estructura de **Lista Simplemente Enlazada**.

El objetivo de este documento es dejar constancia detallada de:

- La problemática abordada y su contexto.
- La estructura de datos empleada y su justificación.
- El razonamiento paso a paso detrás de cada método.
- Los casos de prueba diseñados, tanto positivos como negativos.
- El uso declarado de Inteligencia Artificial durante el proceso.
- El código Java resultante y la integración entre los métodos del equipo.

2. Problemática: Farmacia 'FarmaVida'

En la farmacia **'FarmaVida'** se desea controlar los datos de los medicamentos disponibles. De cada medicamento se conoce:

Campo	Tipo	Descripción
código	String	Identificador único del medicamento (ej. "M01")
nombre	String	Nombre comercial (ej. "Paracetamol")
categoría	Categoría (enum)	Analgésico, Antibiótico, Antiinflamatorio, Vitamina
stock	int	Unidades disponibles en inventario
precio	BigDecimal	Precio unitario en moneda local

Funcionalidades solicitadas (una por integrante)

Inciso	Método	Responsable
a	`MedicamentosAgotados()`	Jorge Luis Reyes Pérez
b	`PromedioPrecioPorCategoria(String categoria)`	Geonel Sánchez
c	`ActualizarStock(String codigo, int cantidadAgregada)`	Merlyn Sosa Borges
d	`ListarPreciosInversos()`	Yaisel Pinos Durán
e	`AplicarAumentoGeneral(double porcentaje)`	Ray Sanjudo Mesa

3. Estructura de Datos Empleada: Lista Simplemente Enlazada

Justificación de la elección

El enunciado asigna explícitamente una **Lista Simplemente Enlazada**. Esta estructura es adecuada para el problema porque:

- Los medicamentos se insertan de forma secuencial y se recorren linealmente.
- No se requieren accesos aleatorios por índice (no hay búsquedas por posición).
- El tamaño del inventario puede crecer y decrecer, y la lista enlazada maneja esto sin redimensionar arreglos.
- Las operaciones principales son recorridos (para promedios, filtros, aumentos), que en una lista enlazada son O(n) y perfectamente aceptables.

4. Proceso de Razonamiento por Método

Nota metodológica

Antes de escribir una sola línea de Java, cada integrante del equipo realizó el razonamiento de su método **a mano y en libreta**, siguiendo una metodología común:

1. **Dibujar la lista** con los nodos del ejemplo del enunciado.
2. **Trazar iteración por iteración** el recorrido, anotando el estado de los punteros.
3. **Identificar casos borde** (lista vacía, un solo nodo, elementos no encontrados).
4. **Escribir el pseudocódigo** antes de pasar a Java.
5. **Verificar con el ejemplo** que el resultado coincida con lo esperado.

Este proceso permitió detectar errores tempranos (como dimensiones incorrectas de arreglos o accesos a punteros nulos) antes de que se convirtieran en bugs de compilación o ejecución.

4.1. `MedicamentosAgotados()` — Jorge Luis Reyes Pérez

Objetivo: Devolver los nombres de los medicamentos cuyo stock sea menor o igual a 5.

Estrategia: Dos pasadas. La primera cuenta cuántos medicamentos cumplen la condición. La segunda llena un arreglo del tamaño exacto.

Razonamiento clave detectado durante el diseño:

Un error común es dimensionar el arreglo con `cantidad` (el total de nodos). Si solo algunos cumplen, el arreglo queda con posiciones `null` al final. La solución correcta es contar primero, y dimensionar con el contador resultante.

Pseudocódigo:

```
PROCEDIMIENTO MedicamentosAgotados()
    SI (inicio == null) ENTONCES RETORNAR arreglo vacío
    contador = 0
    actual = inicio
    CICLO (actual != null)
        SI (actual.getStock() <= 5) ENTONCES contador++
        actual = actual.siguiente
    FIN CICLO
    arreglo = nuevo arreglo de tamaño contador
    i = 0
    actual = inicio
    CICLO (actual != null)
        SI (actual.getStock() <= 5)
            arreglo[i] = actual.getNombre()
            i++
        FIN SI
        actual = actual.siguiente
    FIN CICLO
    RETORNAR arreglo
FIN PROCEDIMIENTO
```

Resultado esperado con el ejemplo:

`["Amoxicilina", "Ibuprofeno", "Azitromicina"]`

4.2. `PromedioPrecioPorCategoria(String categoria)` — Geonel Sánchez

Objetivo: Devolver el precio promedio de la categoría especificada (0.0 si no existen medicamentos de esa categoría).

Estrategia: Recorrer la lista acumulando suma de precios y conteo. Al final, si el conteo es 0, retornar 0.0; si no, dividir.

Razonamiento clave detectado durante el diseño:

Es imprescindible verificar que el contador sea distinto de cero antes de dividir. El caso de "ningún medicamento de esa categoría" se cubre con esa comprobación, y también cubre la lista vacía.

Pseudocódigo:

```
PROCEDIMIENTO PromedioPrecioPorCategoria(categoria)
    SI (categoria == null) ENTONCES RETORNAR 0.0
    actual = inicio
    cant = 0
    cantTotal = 0.0
    CICLO (actual != null)
        SI (actual.getCategoria().equals(categoria))
            cantTotal += actual.getPrecio()
            cant++
        FIN SI
        actual = actual.siguiente
    FIN CICLO
    SI (cant == 0) ENTONCES RETORNAR 0.0
    RETORNAR cantTotal / cant
FIN PROCEDIMIENTO
```

Resultado esperado con el ejemplo:

Para "ANTIBIÓTICO": `(12.00 + 15.00) / 2 = 13.50`

4.3. `ActualizarStock(String codigo, int cantidadAgregada)` — Merlyn Sosa Borges

Objetivo: Buscar un medicamento por su código y sumarle la cantidad entregada. Devolver `true` si existe, `false` si no.

Estrategia: Recorrer la lista comparando el código. Si se encuentra, actualizar el stock y retornar `true` inmediatamente. Si se termina el recorrido sin éxito, retornar `false`.

Razonamiento clave detectado durante el diseño:

El retorno booleano tiene tres significados posibles en este método:

1. El medicamento no existe.
2. La cantidad era negativa (rechazada).
3. El código era `null`.

Para evitar ambigüedad, en cada caso se imprime un mensaje informativo antes de retornar.

Pseudocódigo:

```
PROCEDIMIENTO ActualizarStock(codigo, cantidadAgregada)
    SI (codigo == null)
        ESCRIBIR "El código es null, se rechaza la solicitud"
        RETORNAR false
    FIN SI
    SI (cantidadAgregada < 0)
        ESCRIBIR "La cantidad es negativa, se rechaza la solicitud"
        RETORNAR false
    FIN SI
    actual = inicio
    CICLO (actual != null)
        SI (actual.getCodigo().equals(codigo))
            aumento = actual.getStock() + cantidadAgregada
            actual.setStock(aumento)
            RETORNAR true
        FIN SI
        actual = actual.siguiente
    FIN CICLO
    ESCRIBIR "Medicamento no encontrado"
    RETORNAR false
FIN PROCEDIMIENTO
```

Resultado esperado con el ejemplo:

`ActualizarStock("M02", 10)` → `true`, stock de M02 pasa de 3 a 13.

4.4. `ListarPreciosInversos()` — Yaisel Pinos Durán

Objetivo: Devolver la lista de precios en orden inverso al que fueron ingresados.

Estrategia elegida: patrón mutar–extraer–restaurar.

El método se apoya en un auxiliar privado `invertir()` que da vuelta la lista en su lugar. Los tres pasos son:

1. `invertir()` — la lista queda al revés.

2. Recorrer la lista invertida y llenar un arreglo de precios.
3. `invertir()` otra vez — la lista vuelve a su orden original.

Justificación del patrón:

Este enfoque tiene la ventaja de no necesitar memoria adicional para guardar una copia de la lista. El costo es que la restauración depende de que la segunda llamada a `invertir()` se ejecute correctamente.

Razonamiento clave detectado durante el diseño:

- El arreglo sí se dimensiona con `cantidad`, a diferencia de `MedicamentosAgotados()`. La razón es que **todos** los nodos aportan un precio al resultado, sin filtros.
- El algoritmo de inversión no necesita guard clauses: la lista vacía y la de un solo nodo caen naturalmente en la lógica general del ciclo.
- La línea `siguiente = actual.siguiente` debe leerse **al inicio** del cuerpo del ciclo, no al final, para no intentar acceder a `.siguiente` de un `null` cuando la lista termina.

Pseudocódigo del auxiliar:

```
PROCEDIMIENTO invertir()
    actual = inicio
    anterior = null
    CICLO (actual != null)
        siguiente = actual.siguiente
        actual.siguiente = anterior
        anterior = actual
        actual = siguiente
    FIN CICLO
    inicio = anterior
FIN PROCEDIMIENTO
```

Pseudocódigo del método público:

```
PROCEDIMIENTO ListarPreciosInversos()
    invertir()
    arreglo = nuevo arreglo de tamaño cantidad
    i = 0
    actual = inicio
    CICLO (actual != null)
        arreglo[i] = actual.getPrecio()
        i++
        actual = actual.siguiente
    FIN CICLO
    invertir()
    RETORNAR arreglo
FIN PROCEDIMIENTO
```

Verificación con el ejemplo:

Lista original: `[2.50] → [12.00] → [4.50] → [5.00] → [15.00]`

1. Primera inversión: `[15.00] → [5.00] → [4.50] → [12.00] → [2.50]`
2. Recorrido y llenado: `[15.00, 5.00, 4.50, 12.00, 2.50]`
3. Segunda inversión: `[2.50] → [12.00] → [4.50] → [5.00] → [15.00]` (restaurada)
4. Retorno: `[15.00, 5.00, 4.50, 12.00, 2.50]`

Resultado esperado con el ejemplo:

`[15.00, 5.00, 4.50, 12.00, 2.50]` — coincide con el enunciado.

4.5. `AplicarAumentoGeneral(double porcentaje)` — Ray Sanjudo Mesa

Objetivo: Incrementar el precio de todos los medicamentos en el porcentaje dado y devolver cuántos se modificaron.

Estrategia: Recorrer la lista, aplicar la fórmula `precio * (1 + porcentaje)` a cada nodo, contar cuántos se modifican y retornar el contador.

Razonamiento clave detectado durante el diseño:

- El porcentaje se recibe como **fracción decimal** (`0.25` = 25%). Esta convención se documenta explícitamente para evitar ambigüedad.
- Los porcentajes negativos se rechazan con un mensaje y retorno `0`.
- El porcentaje `0.0` se rechaza también porque no produce cambios reales.
- El retorno `0` tiene tres significados posibles: lista vacía, porcentaje negativo, porcentaje cero.

Pseudocódigo:

```
PROCEDIMIENTO AplicarAumentoGeneral(porcentaje)
    SI (porcentaje < 0)
        ESCRIBIR "El porcentaje no puede ser negativo"
        RETORNAR 0
    FIN SI
    SI (porcentaje == 0.0)
        ESCRIBIR "No hubo cambios: el porcentaje es 0"
        RETORNAR 0
    FIN SI
    actual = inicio
    contador = 0
    CICLO (actual != null)
        nuevoPrecio = actual.getPrecio() * (1 + porcentaje)
        actual.setPrecio(nuevoPrecio)
        contador++
        actual = actual.siguiente
    FIN CICLO
    RETORNAR contador
FIN PROCEDIMIENTO
```

Resultado esperado con el ejemplo (aumento del 10%):

Medicamento	Precio antes	Precio después
M01	2.50	2.75
M02	12.00	13.20
M03	4.50	4.95
M04	5.00	5.50
M05	15.00	16.50

Retorno: 5

5. Casos de Prueba

5.1. Casos positivos (flujos normales)

#	Método	Entrada	Resultado esperado
1	`MedicamentosAgotados()`	Lista con 5 medicamentos	`["Amoxicilina", "Ibuprofeno", "Azitromicina"]`
2	`PromedioPrecioPorCategoria("ANTIBIÓTICO")`	Lista con 5 medicamentos	`13.50`
3	`PromedioPrecioPorCategoria("ANALGÉSICO")`	Lista con 5 medicamentos	`2.50`
4	`ActualizarStock("M02", 10)`	Lista con 5 medicamentos	`true`, stock M02 = 13
5	`ActualizarStock("M03", 0)`	Lista con 5 medicamentos	`true`, stock M03 = 2 (sin cambio)
6	`ListarPreciosInversos()`	Lista con 5 medicamentos	`[15.00, 5.00, 4.50, 12.00, 2.50]`
7	`ListarPreciosInversos()`	Lista con 1 medicamento	`[7.25]`
8	`AplicarAumentoGeneral(0.10)`	Lista con 5 medicamentos	`5`, precios aumentados

5.2. Casos negativos (manejo de errores y bordes)

#	Método	Entrada	Resultado esperado
1	`MedicamentosAgotados()`	Lista vacía	`[]`
2	`MedicamentosAgotados()`	Ninguno con stock ≤ 5	`[]`
3	`PromedioPrecioPorCategoria("CATEGORIA_INEXISTENTE")`	Lista con 5 medicamentos	`0.0`
4	`PromedioPrecioPorCategoria(null)`	Lista con 5 medicamentos	`0.0`
5	`PromedioPrecioPorCategoria(...)`	Lista vacía	`0.0`
6	`ActualizarStock("M99", 10)`	Lista con 5 medicamentos	`false`
7	`ActualizarStock("M02", -5)`	Lista con 5 medicamentos	`false`, stock sin cambios
8	`ActualizarStock(null, 10)`	Lista con 5 medicamentos	`false`
9	`ActualizarStock("M01", 10)`	Lista vacía	`false`
10	`ListarPreciosInversos()`	Lista vacía	`[]`
11	`AplicarAumentoGeneral(-0.10)`	Lista con 5 medicamentos	`0`, precios sin cambios
12	`AplicarAumentoGeneral(0.0)`	Lista con 5 medicamentos	`0`, precios sin cambios
13	`AplicarAumentoGeneral(0.10)`	Lista vacía	`0`

5.3. Casos borde adicionales

#	Método	Entrada	Resultado esperado
1	`MedicamentosAgotados()`	Todos con stock ≤ 5	Arreglo con todos los nombres
2	`ListarPreciosInversos()`	Lista con 1 nodo	Arreglo de 1 precio, lista restaurada
3	`AplicarAumentoGeneral(0.10)`	Lista con 1 nodo	`1`, precio aumentado
4	`PromedioPrecioPorCategoria("VITAMINA")`	Un solo medicamento de esa categoría	Precio de ese medicamento

6. Declaración de Uso de Inteligencia Artificial

En cumplimiento con lo solicitado en la sección de indicaciones generales del examen, el equipo declara de forma explícita y honesta el uso de herramientas de Inteligencia Artificial durante el desarrollo del proyecto.

Herramientas utilizadas

- **Asistente conversacional (tipo Deepseek/Claude)** como apoyo durante el análisis y la documentación.

Partes específicas donde se empleó IA

Fase del trabajo	Uso de IA	Detalle
Análisis de la problemática	Apoyo en la interpretación del enunciado	Se usó IA para aclarar ambigüedades del enunciado y discutir posibles interpretaciones de los métodos. Las decisiones finales fueron tomadas por los integrantes.
Razonamiento de la lógica	Diálogo socrático guiado	Los integrantes escribieron la lógica a mano primero, y la IA funcionó como interlocutor para detectar errores y validar el razonamiento (por ejemplo, el bug del arreglo mal dimensionado en `MedicamentosAgotados()`).
Diseño de casos de prueba	Sugerencia de escenarios borde	La IA ayudó a enumerar casos negativos que se podían pasar por alto (lista vacía, categoría null, código null, cantidad cero, porcentaje cero).
Redacción de la documentación	Revisión de claridad y consistencia	La IA revisó los borradores de documentación en Markdown, detectó contradicciones internas y ayudó a estructurar los informes de cada método.
Revisión de código	Verificación de traducción pseudocódigo → Java	La IA verificó que el código Java de los cinco métodos del equipo fuera fiel al pseudocódigo diseñado y se comportara correctamente en los casos de prueba, incluidos los casos borde.

Lo que NO se hizo con IA

- **No se generó código sin comprensión:** cada línea de Java pasó por el razonamiento manual previo en libreta.

- **No se delegó la toma de decisiones de diseño:** las decisiones sobre convenciones (fracción decimal vs. puntos porcentuales, qué se considera "modificado", qué retorna `false`/`0`) fueron discutidas y aprobadas por el equipo.
- **No se usó IA para generar la exposición oral:** el dominio del tema es responsabilidad directa de cada integrante.

Principio adoptado

El equipo asumió la IA como una **herramienta de apoyo pedagógico**, no como un sustituto del aprendizaje. El objetivo fue que cada integrante pudiera defender técnicamente su método durante la exposición oral, comprendiendo el por qué de cada decisión de diseño.

7. Reflexión sobre el Proceso de Trabajo

Metodología adoptada

El equipo adoptó un enfoque de **razonamiento primero, código después**. Antes de escribir una sola línea de Java, cada integrante:

1. Dibujó la lista de prueba en papel.
2. Trazó iteración por iteración el estado de los punteros.
3. Escribió el pseudocódigo.
4. Identificó casos borde.
5. Verificó el resultado contra el ejemplo del enunciado.
6. Solo entonces tradujo a Java.

Aprendizajes clave

- **Los casos borde son donde se aprende más.** La lista vacía, el nodo único, el elemento no encontrado y los parámetros `null` revelaron errores de diseño que no se veían en el caso "normal".
- **No todos los métodos necesitan guard clauses.** `invertir()` y `PromedioPrecioPorCategoria()` cubren sus casos borde naturalmente con su lógica general, sin ramas especiales. En cambio, `ActualizarStock()` sí necesita validaciones explícitas.
- **El retorno booleano o numérico puede tener múltiples significados.** Documentar cada uno con un mensaje informativo evita que el llamador interprete mal el resultado.
- **La elección de tipos importa.** Usar `BigDecimal` en vez de `double` para precios evita errores de precisión que en un sistema real serían inaceptables.

8. Conclusiones

El desarrollo del Ejercicio 1 permitió al equipo:

- Comprender en profundidad la estructura de **Lista Simplemente Enlazada** y sus operaciones básicas.
- Practicar un método de trabajo riguroso que separa el razonamiento manual de la implementación en código.

- Aprender a identificar y manejar casos borde que no son evidentes en una primera lectura del enunciado.
- Documentar el proceso completo de análisis, diseño e implementación de forma clara y honesta.
- Utilizar herramientas de Inteligencia Artificial de manera ética y declarada, como apoyo al aprendizaje y no como sustituto de él.